Learn JavaScript: Beginner's Course

Lesson 8: DOM Manipulation Basics

DOM Manipulation Basics - Study Notes

Key Concepts

• **Selecting DOM elements** – using methods like `document.getElementById`, `document.querySelector`, and `document.querySelectorAll`.

• **Modifying content** – changing text (`textContent`, `innerText`) and HTML (`innerHTML`).

• **Handling events** – attaching listeners (e.g., `addEventListener('click', handler)`) and responding to user actions.

• **Updating attributes & styles** – altering `className`, `style`, and other element attributes dynamically.

• **Traversing the DOM** – navigating parent, child, and sibling relationships when needed.

Summary

The Document Object Model (DOM) provides a programmatic interface for interacting with HTML documents. By selecting elements with selectors or ID based methods, you can read or change their content, attributes, and styles. Event handling lets you respond to user interactions such as clicks, key presses, or form submissions. Mastering these basics enables you to create dynamic, interactive web pages without reloading the page.

In practice, you'll often combine selection and modification: find a button, attach a click listener, and inside the handler update a paragraph's text or toggle a CSS class. Understanding the difference between `textContent` (safe, text only) and `innerHTML` (allows HTML but can introduce XSS risks) is crucial for writing secure code. Finally, using `addEventListener` rather than inline HTML attributes keeps your JavaScript separate from markup, improving maintainability.

Important Points

• Use `document.querySelector(selector)` for the first match; `querySelectorAll` returns a NodeList (array like) of all matches.

• Prefer `textContent` or `innerText` for plain text updates; reserve `innerHTML` when you truly need to inject HTML.

• Always attach event listeners with `element.addEventListener('eventType', callback)`; avoid inline `onclick` attributes for cleaner code.

• The `event` object passed to handlers contains useful info like `event.target` (the element that triggered the event).

• Changing CSS classes via `element.classList.add/remove/toggle` is safer and more performant than directly manipulating `element.style`.

• Remember that scripts placed in the `<head>` run before the DOM is fully loaded; wrap DOM accessing code in `DOMContentLoaded` or place scripts at the end of `<body>`.

Examples

Example 1: Changing Text on Button Click

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>DOM Demo</title>
  <style>
    #output { font-weight: bold; margin-top: 1rem; }
  </style>
</head>
<body>
  <button id="changeBtn">Change Text</button>
  <p id="output">Original text</p>
  <script>
    // Select elements
    const btn = document.getElementById('changeBtn');
    const output = document.getElementById('output');
    // Define click handler
    function handleClick() {
      output.textContent = 'Text has been changed!';
    }
    // Attach listener
    btn.addEventListener('click', handleClick);
  </script>
</body>
</html>
```

Explanation:

- `getElementById` grabs the button and paragraph.
- The `handleClick` function updates the paragraph's `textContent`.
- `addEventListener` links the click event to the handler, so each click changes the text.

Example 2: Toggling a CSS Class

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Class Toggle</title>
  <style>
    .highlight { background-color: yellow; padding: 0.5rem; }
    .box { width: 100px; height: 100px; background-color: lightblue; margin: 1rem; }
```

```
  </style>
</head>
<body>
  <div id="myBox" class="box"></div>
  <button id="toggleBtn">Toggle Highlight</button>
  <script>
    const box = document.getElementById('myBox');
    const btn = document.getElementById('toggleBtn');
    btn.addEventListener('click', () => {
      box.classList.toggle('highlight'); // adds if absent, removes if present
    });
  </script>
</body>
</html>
```

Explanation:

- `classList.toggle` efficiently adds the `highlight` class on the first click and removes it on the next, giving a visual toggle effect without manually checking the class state.

Quick Review

1. Which method returns a **NodeList** of all elements matching a CSS selector?
   Answer: `document.querySelectorAll(selector)`
2. What is the safest property to use when you only need to change the plain text inside an element?
   Answer: `textContent` (or `innerText`)
3. How do you attach a click event listener to an element without using inline HTML attributes?
   Answer: `element.addEventListener('click', callbackFunction)`
4. When should you prefer `classList.add/remove/toggle` over directly setting `element.style`?
   Answer: When you want to apply predefined CSS classes, keep styling in stylesheets, and benefit from easier maintenance and performance.
5. Why is it important to run DOM manipulating code after the DOM is fully loaded?
   Answer: Elements may not exist yet if the script runs before the HTML is parsed, leading to `null` references and errors.

Further Reading

- **Event Delegation** – handling events at a parent level to manage dynamic content efficiently.
- **Using `MutationObserver`** – reacting to changes in the DOM tree.
- **DOM Performance Tips** – minimizing reflows and repaints, using `requestAnimationFrame` for animations.
- **Security Considerations** – avoiding XSS when using `innerHTML` and sanitizing user input.
- **Modern Alternatives** – exploring libraries/frameworks (e.g., React, Vue) that abstract direct DOM manipulation while still relying on the same underlying concepts.